

```
In [65]: import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay, accuracy_score

from qiskit_aer import Aer
from qiskit_algorithms import VQE
from qiskit.circuit.library import ZZFeatureMap, EfficientSU2
from qiskit_machine_learning.algorithms import VQC
from qiskit_algorithms.optimizers import COBYLA, SPSA
```

```
In [66]: iris = load_iris()

X = iris.data      # (150, 4)
y = iris.target    # 3 classes
```

```
In [67]: scaler = MinMaxScaler(feature_range=(0, np.pi))
X = scaler.fit_transform(X)
```

```
In [68]: X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
In [69]: y_train_0 = (y_train == 0).astype(int)
y_train_1 = (y_train == 1).astype(int)
y_train_2 = (y_train == 2).astype(int)
```

```
In [70]: from qiskit.primitives import Sampler
from qiskit.primitives import StatevectorSampler
#sampler = Sampler()
sampler = StatevectorSampler()
```

```
In [71]: def train_vqc_epochs(X_train, y_train, X_test, y_test, epochs=20):

    feature_map = ZZFeatureMap(feature_dimension=4, reps=2)
    ansatz = EfficientSU2(num_qubits=4, reps=2)

    vqc = VQC(
        feature_map=feature_map,
        ansatz=ansatz,
        optimizer=COBYLA(maxiter=100, tol=1e-3), # small steps
        #optimizer=SPSA(),
        sampler=sampler
    )

    loss_history = []
    train_acc = []
    test_acc = []

    for epoch in range(epochs):

        vqc.fit(X_train, y_train) # incremental training

        train_score = vqc.score(X_train, y_train)
        test_score = vqc.score(X_test, y_test)

        loss = 1 - train_score

        loss_history.append(loss)
        train_acc.append(train_score)
        test_acc.append(test_score)

        print(f"Epoch {epoch+1}: Train={train_score:.3f}, Test={test_score:.3f}")

    return vqc, loss_history, train_acc, test_acc
```

```
In [72]: vqc_0, loss0, train0, test0 = train_vqc_epochs(X_train, y_train_0, X_test, y_test)
vqc_1, loss1, train1, test1 = train_vqc_epochs(X_train, y_train_1, X_test, y_test)
vqc_2, loss2, train2, test2 = train_vqc_epochs(X_train, y_train_2, X_test, y_test)
```

No gradient function provided, creating a gradient function. If your Sampler requires transpilation, please provide a pass manager.

```
Epoch 1: Train=0.758, Test=0.133
Epoch 2: Train=0.775, Test=0.200
Epoch 3: Train=0.750, Test=0.367
Epoch 4: Train=0.717, Test=0.433
Epoch 5: Train=0.725, Test=0.233
Epoch 6: Train=0.725, Test=0.300
Epoch 7: Train=0.775, Test=0.267
Epoch 8: Train=0.700, Test=0.267
Epoch 9: Train=0.708, Test=0.233
Epoch 10: Train=0.575, Test=0.300
Epoch 11: Train=0.700, Test=0.233
Epoch 12: Train=0.708, Test=0.267
Epoch 13: Train=0.742, Test=0.267
Epoch 14: Train=0.667, Test=0.233
Epoch 15: Train=0.725, Test=0.267
Epoch 16: Train=0.742, Test=0.267
Epoch 17: Train=0.625, Test=0.267
Epoch 18: Train=0.642, Test=0.267
Epoch 19: Train=0.683, Test=0.233
```

No gradient function provided, creating a gradient function. If your Sampler requires transpilation, please provide a pass manager.

```
Epoch 20: Train=0.742, Test=0.200
Epoch 1: Train=0.683, Test=0.433
Epoch 2: Train=0.675, Test=0.467
Epoch 3: Train=0.600, Test=0.500
Epoch 4: Train=0.675, Test=0.433
Epoch 5: Train=0.633, Test=0.500
Epoch 6: Train=0.692, Test=0.433
Epoch 7: Train=0.675, Test=0.367
Epoch 8: Train=0.633, Test=0.467
Epoch 9: Train=0.692, Test=0.367
Epoch 10: Train=0.700, Test=0.400
Epoch 11: Train=0.667, Test=0.400
Epoch 12: Train=0.617, Test=0.467
Epoch 13: Train=0.675, Test=0.467
Epoch 14: Train=0.692, Test=0.467
Epoch 15: Train=0.642, Test=0.433
Epoch 16: Train=0.650, Test=0.400
Epoch 17: Train=0.692, Test=0.400
Epoch 18: Train=0.700, Test=0.400
Epoch 19: Train=0.633, Test=0.500
```

No gradient function provided, creating a gradient function. If your Sampler requires transpilation, please provide a pass manager.

```
Epoch 20: Train=0.675, Test=0.433
Epoch 1: Train=0.683, Test=0.300
Epoch 2: Train=0.625, Test=0.267
Epoch 3: Train=0.633, Test=0.333
Epoch 4: Train=0.708, Test=0.300
Epoch 5: Train=0.775, Test=0.333
Epoch 6: Train=0.658, Test=0.367
Epoch 7: Train=0.742, Test=0.267
Epoch 8: Train=0.567, Test=0.233
Epoch 9: Train=0.650, Test=0.367
Epoch 10: Train=0.592, Test=0.233
Epoch 11: Train=0.650, Test=0.267
Epoch 12: Train=0.625, Test=0.333
Epoch 13: Train=0.675, Test=0.300
Epoch 14: Train=0.583, Test=0.300
Epoch 15: Train=0.675, Test=0.300
Epoch 16: Train=0.633, Test=0.433
Epoch 17: Train=0.633, Test=0.433
Epoch 18: Train=0.650, Test=0.267
Epoch 19: Train=0.667, Test=0.467
Epoch 20: Train=0.650, Test=0.200
```

```
In [73]: def predict_multiclass(X):

        p0 = vqc_0.predict(X)
        p1 = vqc_1.predict(X)
        p2 = vqc_2.predict(X)

        preds = np.vstack([p0, p1, p2]).T

        return np.argmax(preds, axis=1)
```

```
In [74]: y_pred = predict_multiclass(X_test)

acc = accuracy_score(y_test, y_pred)
print("Final Test Accuracy:", acc)
```

Final Test Accuracy: 0.36666666666666664

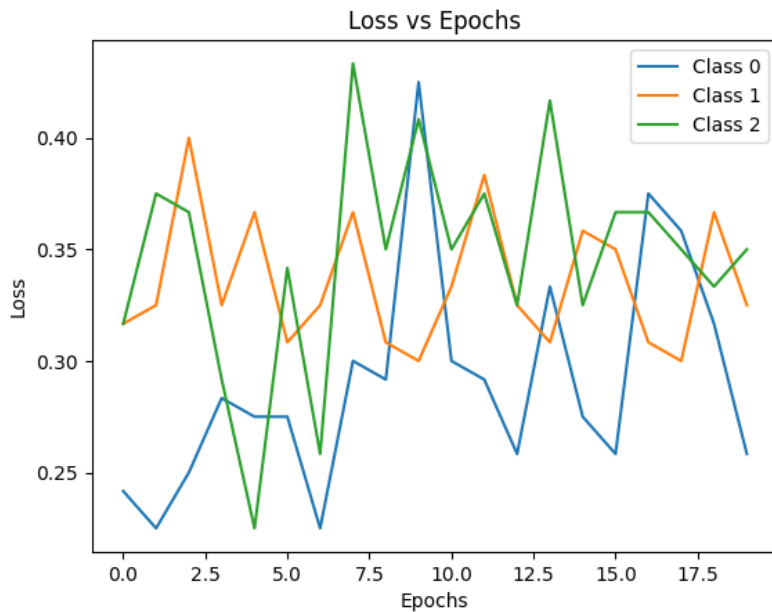
```
In [75]: #Plot Loss vs Epochs
plt.figure()

plt.plot(loss0, label="Class 0")
plt.plot(loss1, label="Class 1")
```

```
plt.plot(loss2, label="Class 2")

plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.title("Loss vs Epochs")
plt.legend()

plt.show()
```

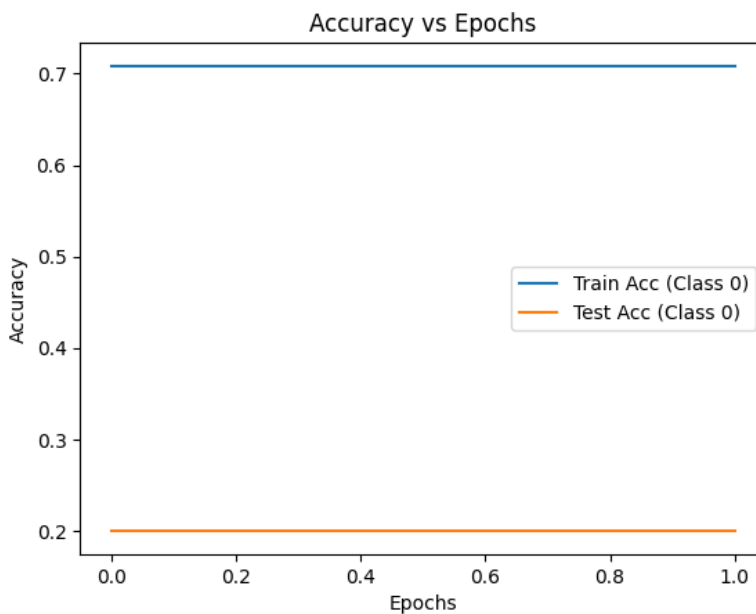


```
In [76]: #Plot Accuracy vs Epochs
plt.figure()

plt.plot(train_acc0, label="Train Acc (Class 0)")
plt.plot(test_acc0, label="Test Acc (Class 0)")

plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.title("Accuracy vs Epochs")
plt.legend()

plt.show()
```

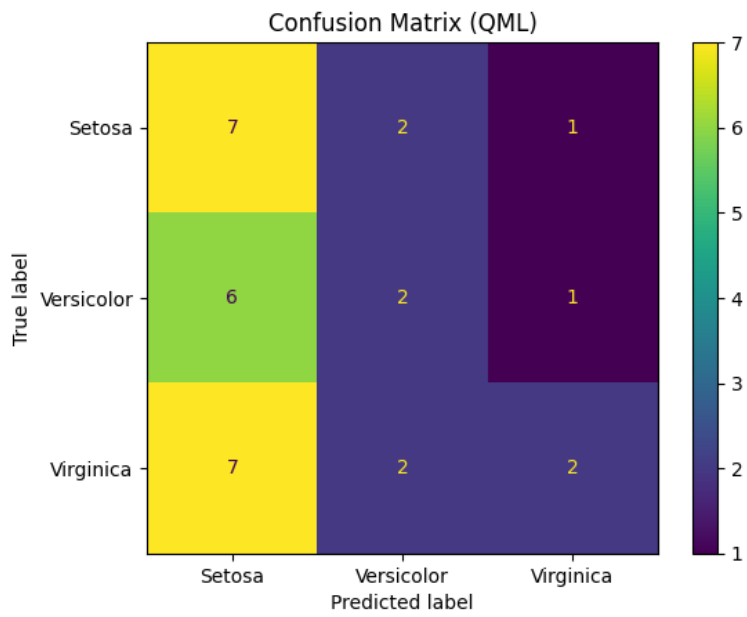


```
In [77]: #Confusion Matrix (Heatmap)
cm = confusion_matrix(y_test, y_pred)

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["Setosa", "Versicolor", "Virginica"]
)

disp.plot()
plt.title("Confusion Matrix (QML)")
```

```
plt.show()
```



```
In [ ]:
```